# Chunfan **Yao**

Undergraduate Researcher · AI Agents and Verification

*Berkeley, CA (Exchange) / Guangzhou, China*

(+86) 186-2061-1072 | cfyao_berk@berkeley.edu | https://ferrari-72.github.io/ | Ferrari-72


## Summary


HKUST(GZ) AI undergraduate and Fall 2026 visiting student at UC Berkeley. I build evaluation environments and executable checks for coding and research agents. I am interested in applying these methods to agent security and program analysis.


## Education

**The Hong Kong University of Science and Technology (Guangzhou)** — *Guangzhou, China*

B.Eng. in Artificial Intelligence — *Sep. 2024 – Present*

**University of California, Berkeley** — *Berkeley, CA*

Visiting Undergraduate (Exchange) — *Aug. 2026 – Dec. 2026*

## Research Experience

**Unit-Weight Gadgets for Rydberg-Atom Arrays — Prof. Jinguo Liu's Group, HKUST(GZ)** — *Guangzhou, China*

Undergraduate Research Assistant (Co-First Author) — *2025 – Present*

- Developed a tensor-preserving vertex-split construction that lowers pin degree while retaining the combinatorial behavior required by unit-weight gadgets on triangular Rydberg lattices.
- Built reduced-$\alpha$ tensor checks, lattice-animal enumeration, and port validation in GadgetSearch.jl to test candidate constructions. Co-first-author manuscript in preparation for submission to IEEE TCAD.

**LLM Combinatorial Reasoning Benchmark — Prof. Jinguo Liu's Group, HKUST(GZ)** — *Guangzhou, China*

Research Contributor — *2025 – 2026*

- Turned logical bugs in combinatorial reductions into reproducible agent tasks: built SWE-style and mini-SWE-agent runners with a common interface, concurrent execution, and archived trajectories for problem-reductions-benchmark.
- Used the independent `pred` verifier to check bug certificates instead of accepting an agent's explanation; defined bugs-per-token and bugs-per-dollar metrics to compare finding efficiency.

**AutoResearch for Reduction-Rule Discovery — HKUST(GZ)** — *Guangzhou, China*

Research Contributor — *2026 – Present*

- Designed the problem-finding stage for autonomous reduction-rule discovery: turn open questions into explicit candidate tasks, retain counterexamples, and send proposals to independent checks. This targets the bottleneck between generating rules and choosing worthwhile problems.

**Coding-Agent Evaluation for Linux Kernel State Machines — Tsinghua University** — *China*

Research Intern — *2026*

- Investigated whether coding agents can extract state machines from Linux kernel code and worked on ground-truth examples for evaluation. Final extraction quality was limited, making this a feasibility study rather than a validated extraction pipeline.

## Selected Project

**Reduction Impact Atlas** — *Open Source*

Independent Research Tool — *2026*

- Built a reproducible ranking of 76 open reduction questions using frozen graph data and explicit bridge evidence. Scores estimate conditional reachability; unverified mappings remain labeled provisional. Source code.

## Technical Skills

| | |
|---|---|
| **Languages** | Python, Julia, Git, Linux/CLI, LaTeX |
| **Agent/Eval** | Agent evaluation harnesses, executable verifiers, benchmark design, trajectory analysis |
| **Methods** | Graph algorithms, combinatorial optimization, PyTorch, experiment automation |